Task 3: Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

```
 GNU nano 7.2                                    process_state.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid;

    printf("Before fork():\n");
    printf("PID  = %d\n", getpid());
    printf("PPID = %d\n", getppid());
    printf("State = Parent process is running\n\n");

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    else if (pid == 0) {
        printf("Child Process:\n");
        printf("PID  = %d\n", getpid());
        printf("PPID = %d\n", getppid());
        printf("State = Child process is running\n\n");

        sleep(2);

        printf("Child Process After Sleep:\n");
        printf("PID  = %d\n", getpid());
        printf("PPID = %d\n", getppid());
        printf("State = Child process completed\n");

        exit(0);
    }

    else {
        printf("Parent Process:\n");
        printf("PID  = %d\n", getpid());
        printf("PPID = %d\n", getppid());
        printf("State = Parent waiting for child\n\n");

        wait(NULL);

        printf("Parent Process After wait():\n");
        printf("PID  = %d\n", getpid());
        printf("PPID = %d\n", getppid());
        printf("State = Child completed, parent continues\n");
    }

    return 0;
}
```

```
suraj@DESKTOP-BLT5L3R:~$ cd ~
suraj@DESKTOP-BLT5L3R:~$ nano process_state.c
suraj@DESKTOP-BLT5L3R:~$ gcc process_state.c -o process_state
suraj@DESKTOP-BLT5L3R:~$ ./process_state
Before fork():
PID  = 820
PPID = 332
State = Parent process is running

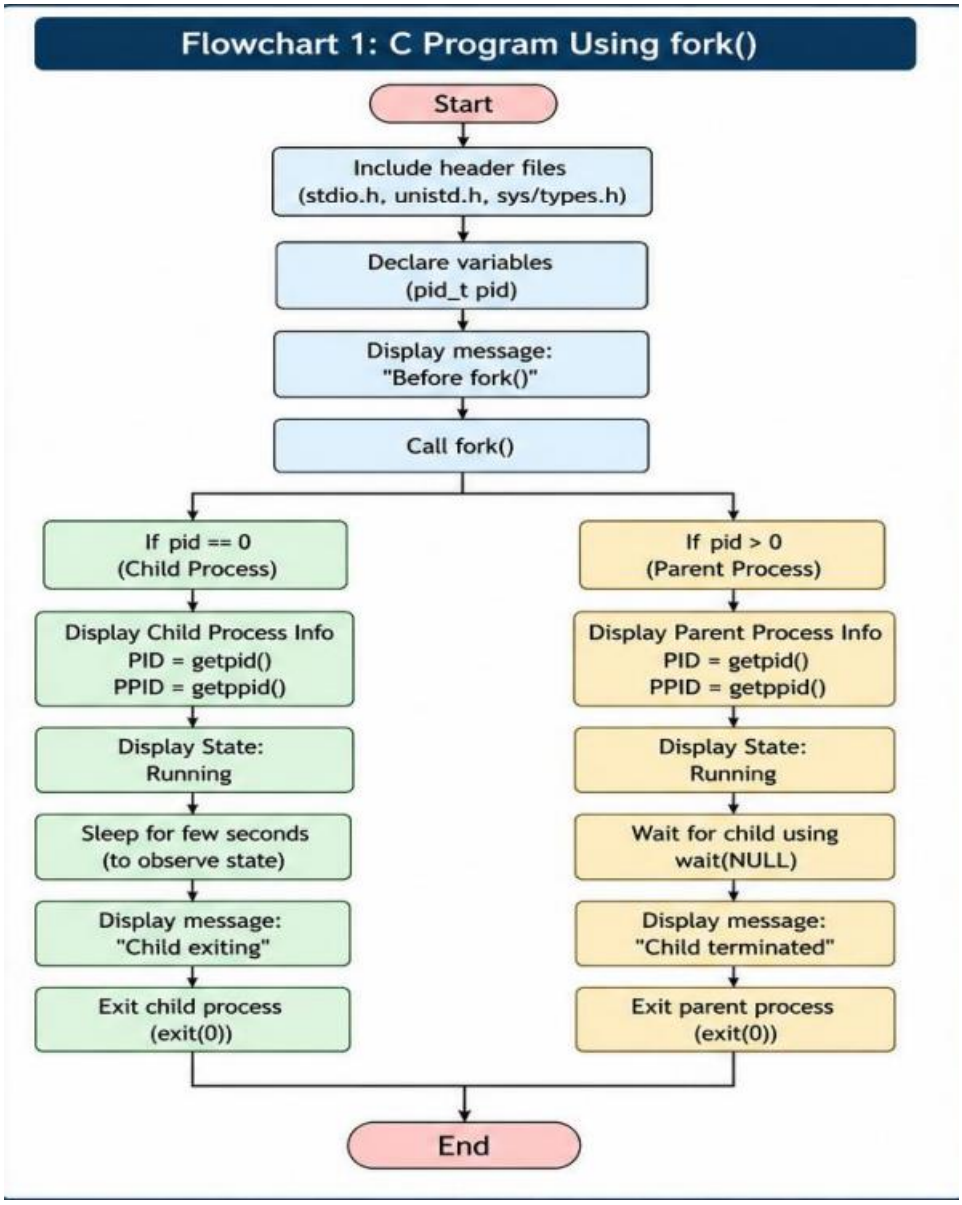
Parent Process:
PID  = 820
PPID = 332
State = Parent waiting for child

Child Process:
PID  = 821
PPID = 820
State = Child process is running

Child Process After Sleep:
PID  = 821
PPID = 820
State = Child process completed

Parent Process After wait():
PID  = 820
PPID = 332
State = Child completed, parent continues
suraj@DESKTOP-BLT5L3R:~$
```

Flow chart:-



Observation:-

1. fork() successfully creates a parent and child process with different PIDs.

2. The child process has the parent's PID as its PPID.

3. Process states change as the parent waits and the child executes.

4. wait() synchronizes the parent with the completion of the child process.

Task 4: Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

```
 GNU nano 7.2                          state.c *
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("Process started\n");
    printf("PID = %d\n", getpid());
    fflush(stdout);

    printf("Running...\n");
    sleep(10);

    printf("Waiting...\n");
    sleep(20);

    printf("Running again...\n");
    sleep(10);

    printf("Process terminating...\n");

    return 0;
}
```

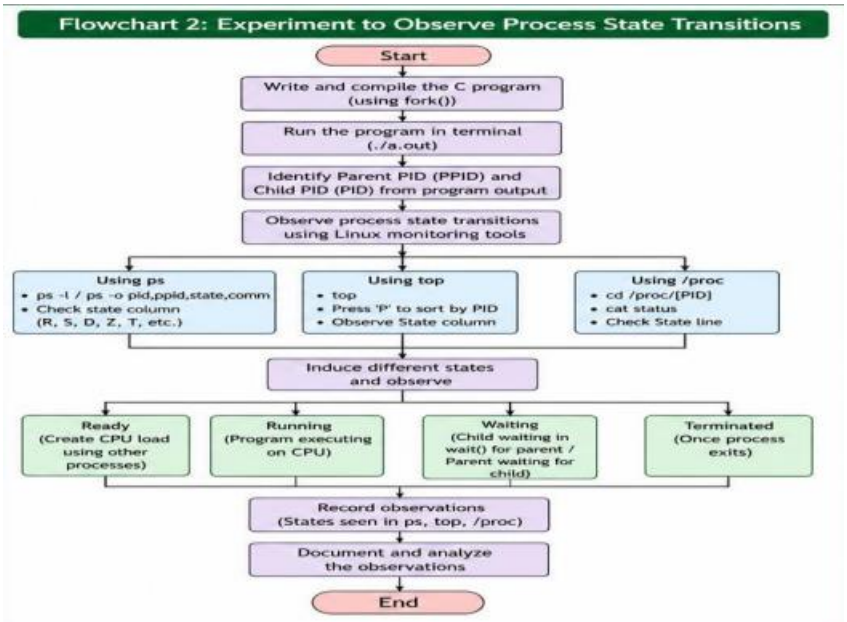
```
raj@DESKTOP-BLT5L3R:~$ ps -o pid,ppid,state,cmd | grep 3456
  3456    3120    R ./state
uraj@DESKTOP-BLT5L3R:~$ top -p 3456
op - 14:23:17 up  2:32,  1 user,  load average: 0.00, 0.00, 0.00
asks:   1 total,   1 running,   0 sleeping,   0 stopped,   0 zombie
Cpu(s):  1.3 us,  0.7 sy,  0.0 ni, 97.4 id,  0.3 wa,  0.0 hi,  0.3 si,  0.0 st
iB Mem :   7392.7 total,   5821.4 free,    984.1 used,    587.2 buff/cache
iB Swap:   2048.0 total,   2048.0 free,      0.0 used.   6408.6 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   3456 suraj    20   0   10540    932    776 R  13.6   0.0   0:00.04 ./state
```

```
suraj@DESKTOP-BLT5L3R:~$ nano state.c
suraj@DESKTOP-BLT5L3R:~$ gcc state.c -o state
suraj@DESKTOP-BLT5L3R:~$ ./state &
[1] 876
suraj@DESKTOP-BLT5L3R:~$ Process started
PID = 876
Running...
Waiting...
Running again...
Process terminating...
```

```
araj@DESKTOP-BLT5L3R:~$ pstree -p
ystemd(1)─┬─agetty(227)
          ├─cron(186)
          ├─dbus-daemon(187)
          ├─init-systemd(Ub(2)─┬─SessionLeader(916)───Relay(922)(918)───ba+
          │                    ├─init(8)───{init}(9)
          │                    ├─login(333)───bash(412)
          │                    └─{init-systemd(Ub}(10)
          ├─rsyslogd(217)─┬─{rsyslogd}(235)
          │               ├─{rsyslogd}(236)
          │               └─{rsyslogd}(237)
          ├─systemd(383)───(sd-pam)(384)
          ├─systemd-journal(63)
          ├─systemd-logind(200)
          ├─systemd-resolve(173)
          ├─systemd-timesyn(177)───{systemd-timesyn}(181)
          ├─systemd-udevd(109)
          ├─unattended-upgr(244)───{unattended-upgr}(287)
          └─wsl-pro-service(203)─┬─{wsl-pro-service}(247)
                                 ├─{wsl-pro-service}(248)
                                 ├─{wsl-pro-service}(249)
                                 ├─{wsl-pro-service}(250)
                                 ├─{wsl-pro-service}(251)
                                 ├─{wsl-pro-service}(253)
                                 └─{wsl-pro-service}(254)
uraj@DESKTOP-BLT5L3R:~$ ps -p 3456
   PID TTY          TIME CMD
uraj@DESKTOP-BLT5L3R:~$
```

Flow chart:



Observation :-

1. The process state can be monitored using ps, top, and /proc.

2. The process changes between running/runnable and waiting/sleeping states during execution.

3. After completion, the process enters the terminated state and disappears from the process table and /proc.

4. Linux represents process states using codes such as R (running/runnable) and S (sleeping/waiting)